

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_e3b53aed-f4e\agent-a111a6f7b5d097c3b.jsonl`
- SHA-256: `27e3c800f7407e8c5ab73d22b1eceb4153f48a5e32d6811f3dbfdfaad489d5ee`
- Source modified: `2026-06-18T02:04:19+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_e3b53aed-f4e`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T02:00:03.299Z)

Review a bug-fix PR in badminton-highlight-indexer. Work READ-ONLY in this git worktree (do NOT modify): C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-unwrap

First run ``git -C "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/scripts-unwrap" diff origin/master..HEAD`` to see the exact change, then read the touched files + related code/tests. PR #213 fixes two standalone scripts (scripts/run_phase3_eval.py, scripts/run_process_and_stitch.py) that called `len()/truthiness` on a raw segmenter Result (Success|Failure dataclass, no `__len__`) ? crashing under the default 'motion' segmenter. Adds backend/results.unwrap_or_failure(result) -> (value, failure) (DRY of the backend/main.py inline unwrap) + tests/test_results.py.

Be adversarial ? try to find a real reason NOT to merge. Concrete, verifiable findings only (cite file:line); no style noise. severity must-fix only for a genuine bug/regression/break. blocks_merge=true ONLY on a real must-fix. Return the structured object.

LENS ? correctness: is the fix logically right? For #212: does swallowing ONLY 'duplicate column name' avoid masking other errors, and does the happy path still bump versions exactly as before? For #213: does unwrap_or_failure handle Success/Failure/bare-list correctly, and is the Failure branch (continue vs return) right for each script?

2. assistant (2026-06-18T02:00:06.419Z)

I'll review PR #213. Let me start by examining the exact change.

3. user (2026-06-18T02:00:07.716Z)

```
diff --git a/backend/results.py b/backend/results.py
index d6b1ea4..75120c5 100644
--- a/backend/results.py
+++ b/backend/results.py
@@ -18,7 +18,7 @@ This module has no dependencies on the rest of the app.
     from __future__ import annotations

     from dataclasses import dataclass, field
     from typing import Generic, TypeVar, List, Optional, Any, Union
+from typing import Generic, TypeVar, List, Optional, Any, Tuple, Union

     T = TypeVar("T")

@@ -58,3 +58,21 @@ class Failure:
     # Result[ValueType] means Success[ValueType] | Failure
     # (the Failure "error" type is fixed for this contract)
```

```

Result = Union[Success[T], Failure]
+
+
+def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
+    """Split a segmenter/provider return into ``(value, failure)`` so a caller branches once.
+
+    - ``Failure`` -> ``(None, failure)``
+    - ``Success(value=v)`` -> ``(v, None)``
+    - a bare value (legacy code that predates this contract and still returns a raw list)
+    -> ``(value, None)``
+
+    This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline; callers
+    that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
+    len()``
+    on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is
+    heterogeneous
+    (Success | Failure | legacy bare value)."""
+    if isinstance(result, Failure):
+        return None, result
+    value = result.value if hasattr(result, "value") else result
+    return value, None
diff --git a/scripts/run_phase3_eval.py b/scripts/run_phase3_eval.py
index 56e21ca..dd1263e 100644
--- a/scripts/run_phase3_eval.py
+++ b/scripts/run_phase3_eval.py
@@ -34,6 +34,7 @@ from backend.config import load_config
from backend.utils.hashing import compute_video_id as get_file_md5 # canonical video_id
# Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
from backend.pipeline.segmenters import segmenter_registry
+from backend.results import unwrap_or_failure
from backend.eval.annotations import load_annotations
from backend.eval.harness import evaluate, format_report_text, report_to_dict
from backend.eval import batch
@@ -142,7 +143,15 @@ def main(argv=None) -> int:
    print(f"[proc] {vid}: segmenting with {args.segmenter}...")
    db.add_video(video_id, video_path, "processed", [])
    t0 = time.time()
-    segs = segmenter.process_video(video_id, video_path,
max_frames=args.max_frames)
+    # process_video returns a Result (Success|Failure) for some segmenters
+    # (e.g. the default `motion`); unwrap before len() so this doesn't crash with
+    # TypeError on a Success dataclass ? mirrors the unwrap in backend/main.py.
+    segs, failure = unwrap_or_failure(
+        segmenter.process_video(video_id, video_path, max_frames=args.max_frames))
+    if failure is not None:
+        print(f"          -> FAILED: {failure.message}")
+        skipped.append((vid, f"segment failed: {failure.message}"))
+        continue
    print(f"          -> {len(segs)} segments in {time.time() - t0:.0f}s")

    gt = load_annotations(csv_path)
diff --git a/scripts/run_process_and_stitch.py b/scripts/run_process_and_stitch.py
index b8ffef6..ea7c2de 100644
--- a/scripts/run_process_and_stitch.py
+++ b/scripts/run_process_and_stitch.py
@@ -16,6 +16,7 @@ from backend.config import load_config
from backend.utils.hashing import compute_video_id
from backend.pipeline.segmenters import segmenter_registry
from backend.pipeline.stitcher.compiler import VideoCompiler
+from backend.results import unwrap_or_failure
from backend.tools.export import format_rally_summary

logger = logging.getLogger(__name__)
@@ -78,7 +79,13 @@ def main():
    return

```

```

        print(f"Using segmenter: {seg_name}", flush=True)
        segmenter = segmenter_cls(db, cfg)
-       segments = segmenter.process_video(video_id, video_path)
+       # process_video returns a Result (Success|Failure) for some segmenters (e.g. `motion`);
+       # unwrap before len()/truthiness so this doesn't crash with TypeError on a Success
+       # dataclass (and so the empty-guard below actually sees the list) ? mirrors
backend/main.py.
+       segments, failure = unwrap_or_failure(segmenter.process_video(video_id, video_path))
+       if failure is not None:
+           print(f"[FAIL] Segmentation failed: {failure.message}")
+           return
        print(f"Successfully indexed {len(segments)} play segments.", flush=True)

    if not segments:
diff --git a/tests/test_results.py b/tests/test_results.py
new file mode 100644
index 0000000..5b50767
--- /dev/null
+++ b/tests/test_results.py
@@ -0,0 +1,39 @@
+"""Unit tests for ``backend.results`` ? the Success/Failure contract + the
+``unwrap_or_failure``
+helper that standalone scripts use to avoid ``len()``-ing a raw ``Success`` dataclass."""
+import pytest
+
+from backend.results import Failure, Success, unwrap_or_failure
+
+def test_unwrap_success_returns_value_and_no_failure():
+    value, failure = unwrap_or_failure(Success(value=[{"start": 1.0}]))
+    assert value == [{"start": 1.0}]
+    assert failure is None
+
+def test_unwrap_failure_returns_failure_and_no_value():
+    f = Failure(message="Could not open video: x.mp4", is_recoverable=False)
+    value, failure = unwrap_or_failure(f)
+    assert value is None
+    assert failure is f
+
+def test_unwrap_bare_value_passes_through():
+    """Legacy segmenters that predate the Result contract still return a bare list ? the helper
+    passes those through unchanged as ``(value, None)`` so a mixed segmenter set all works."""
+    bare = [{"start": 1.0}, {"start": 2.0}]
+    value, failure = unwrap_or_failure(bare)
+    assert value == bare
+    assert failure is None
+
+def test_unwrap_fixes_the_len_on_success_crash():
+    """Regression for the scripts bug: ``len()`` on a raw ``Success`` raises ``TypeError``;
+    unwrapping first yields the underlying list so callers (run_phase3_eval /
+    run_process_and_stitch, which default to the ``motion`` segmenter) can ``len()`` it."""
+    result = Success(value=[1, 2, 3])
+    with pytest.raises(TypeError):
+        len(result) # the original bug (Success has no __len__)
+    segs, failure = unwrap_or_failure(result)
+    assert failure is None
+    assert len(segs) == 3 # fixed

```

4. assistant (2026-06-18T02:00:10.073Z)

Now let me look at the full files and the related code (backend/main.py inline unwrap, results.py base classes).

5. user (2026-06-18T02:00:11.058Z)

```
1      """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3      This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5      The goal is to make it impossible to confuse "no data / no rallies" with "something
6      actually failed"
7      (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
8
9      We use simple dataclasses so callers can use isinstance checks or structural pattern
10     matching.
11
12     This is intentionally small and incremental ? we start by applying it to the segmenter
13     and provider layers (the biggest sources of silent [] returns today).
14
15     The types are generic where it makes sense. Callers can still access .value on Success
16     or .message on Failure.
17
18     This module has no dependencies on the rest of the app.
19     """
20
21     from __future__ import annotations
22
23     from dataclasses import dataclass, field
24     from typing import Generic, TypeVar, List, Optional, Any, Tuple, Union
25
26     T = TypeVar("T")
27
28     @dataclass
29     class Success(Generic[T]):
30         """Represents a successful outcome with an optional list of non-fatal warnings."""
31
32         value: T
33         warnings: List[str] = field(default_factory=list)
34
35         def is_success(self) -> bool:
36             return True
37
38         def unwrap(self) -> T:
39             """Return the value. Only safe to call if you already know it's a Success."""
40             return self.value
41
42     @dataclass
43     class Failure:
44         """Represents a failure with a human-readable message and optional details."""
45
46         message: str
47         details: Optional[dict[str, Any]] = None
48         is_recoverable: bool = False # e.g. missing API key (recoverable) vs internal crash
49
50         def is_success(self) -> bool:
51             return False
52
53         def unwrap(self) -> Any:
54             """Raise to make misuse obvious in code that assumes success."""
55             raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
56
57     # Convenient alias for use in type hints: Result[list[dict], Failure]
58     # Result[ValueType] means Success[ValueType] | Failure
59     # (the Failure "error" type is fixed for this contract)
60     Result = Union[Success[T], Failure]
61
```

```

62
63 def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
64     """Split a segmenter/provider return into ``(value, failure)`` so a caller branches
once.
65
66     - ``Failure``          -> ``(None, failure)``
67     - ``Success(value=v)`` -> ``(v, None)``
68     - a bare value (legacy code that predates this contract and still returns a raw
list)
69     -> ``(value, None)``
70
71     This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline;
callers
72     that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
len()``
73     on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is
heterogeneous
74     (Success | Failure | legacy bare value)."""
75     if isinstance(result, Failure):
76         return None, result
77     value = result.value if hasattr(result, "value") else result
78     return value, None
79

```

6. user (2026-06-18T02:00:12.003Z)

```

1     """Phase 3 driver: batch process + evaluate a collector export against ground truth.
2
3     Walks the `manifest.json` produced by sports-data-collector's `curate export`,
4     processes each video through a segmenter (so detections land in the indexer DB),
5     scores each against its ground-truth rally CSV, and prints per-video plus
6     corpus-aggregate precision/recall/F1, mean IoU, and boundary error.
7
8     Examples
9     -----
10    Free CPU baseline on a single video (validate the loop):
11        python run_phase3_eval.py --manifest ../sports-data-
collector/data/eval_export/manifest.json \
12        --segmenter motion --only shuttlest_1
13
14    Full corpus with the two-stage detector (needs GEMINI_API_KEY; costs API calls):
15        python run_phase3_eval.py --manifest ../manifest.json --segmenter yolo_hybrid \
16        --json output/phase3_report.json
17
18    Re-score only (videos already processed), no re-segmentation:
19        python run_phase3_eval.py --manifest ../manifest.json --score-only
20    """
21
22    import os
23    import sys
24    import json
25    import time
26    import argparse
27    import logging
28
29    from dotenv import load_dotenv
30    load_dotenv() # make GEMINI_API_KEY (etc.) available to the AI-backed segmenters
31
32    from backend.infrastructure.database import Database
33    from backend.config import load_config
34    from backend.utils.hashing import compute_video_id as get_file_md5 # canonical video_id
35    # Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
36    from backend.pipeline.segmenters import segmenter_registry
37    from backend.results import unwrap_or_failure
38    from backend.eval.annotations import load_annotations

```

```

39     from backend.eval.harness import evaluate, format_report_text, report_to_dict
40     from backend.eval import batch
41
42     logger = logging.getLogger(__name__)
43
44
45     def build_parser() -> argparse.ArgumentParser:
46         p = argparse.ArgumentParser(
47             description="Batch process + evaluate a collector export manifest (Phase 3).",
48             formatter_class=argparse.RawDescriptionHelpFormatter,
49         )
50         p.add_argument("--manifest", required=True, help="Path to the collector's
manifest.json.")
51         p.add_argument("--videos-root", default=None,
52             help="Root for resolving manifest relative local_paths "
53             "(default: the collector repo root, two levels above the
manifest).")
54         p.add_argument("--segmenter", default="motion",
55             help="Segmenter to run (default: motion ? free/CPU). "
56             "yolo_hybrid/gemini need GEMINI_API_KEY and cost API calls.")
57         p.add_argument("--stage", choices=("candidates", "final", "both", "auto"),
58             default="auto",
59             help="Which prediction stage(s) to score (default: auto by
segmenter).")
60         p.add_argument("--detector", default=None, help="Restrict candidate scoring to this
detector.")
61         p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
62         p.add_argument("--only", action="append", default=None,
63             help="Only this video_id (repeatable).")
64         p.add_argument("--limit", type=int, default=None, help="Process at most N videos.")
65         p.add_argument("--max-frames", type=int, default=None,
66             help="Cap segmenter frames (fast validation).")
67         p.add_argument("--score-only", action="store_true",
68             help="Skip segmentation; only score detections already in the DB.")
69         p.add_argument("--reprocess", action="store_true",
70             help="Re-run segmentation even if the video already has segments.")
71         p.add_argument("--db", default=None, help="Indexer DB path (default:
output/<db_name>).")
72         p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report
here.")
73         p.add_argument("--show-failures", action="store_true", help="List missed/spurious
rallies per video.")
74         return p
75
76     def main(argv=None) -> int:
77         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
78         args = build_parser().parse_args(argv)
79
80         manifest_path = os.path.abspath(args.manifest)
81         if not os.path.exists(manifest_path):
82             print(f"ERROR: manifest not found: {manifest_path}", file=sys.stderr)
83             return 2
84         manifest_dir = os.path.dirname(manifest_path)
85         videos_root = args.videos_root or os.path.dirname(os.path.dirname(manifest_dir))
86
87         with open(manifest_path) as f:
88             manifest = json.load(f)
89             entries = manifest.get("eval_sets", [])
90
91         cfg = load_config() # typed AppConfig (same object the live pipeline feeds
segmenters)
92         db_path = args.db or os.path.join(cfg.output.output_dir, cfg.output.db_name)
93         os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)

```

```

94         db = Database(db_path)
95
96         stages = batch.default_stages_for(args.segmenter, args.stage)
97
98         # Resolve the segmenter class up front (unless purely scoring).
99         segmenter = None
100         if not args.score_only:
101             seg_cls = segmenter_registry.get(args.segmenter)
102             if not seg_cls:
103                 print(f"ERROR: segmenter '{args.segmenter}' not found. "
104                       f"Available: {list(segmenter_registry.list_available().keys())}",
105                       file=sys.stderr)
106                 return 2
107             segmenter = seg_cls(db, cfg)
108
109         # Filter entries.
110         targets = []
111         for e in entries:
112             if args.only and e["video_id"] not in args.only:
113                 continue
114             targets.append(e)
115         if args.limit:
116             targets = targets[: args.limit]
117
118         per_video = []
119         skipped = []
120         print(f"Phase 3: {len(targets)} target(s) | segmenter={args.segmenter} |
121               stages={stages} | db={db_path}\n")
122
123         for e in targets:
124             vid = e["video_id"]
125             video_path = batch.resolve_video_path(e.get("local_path"), videos_root)
126             csv_path = os.path.join(manifest_dir, e["csv"])
127
128             if not video_path or not os.path.exists(video_path):
129                 skipped.append((vid, "no video file"))
130                 print(f"[skip] {vid}: no video file ({e.get('local_path')})")
131                 continue
132             if not os.path.exists(csv_path):
133                 skipped.append((vid, "no GT csv"))
134                 print(f"[skip] {vid}: GT csv missing ({csv_path})")
135                 continue
136
137             video_id = get_file_md5(video_path)
138
139             # Segment unless score-only or already present (and not --reprocess).
140             if not args.score_only:
141                 already = db.query_play_segments(video_id, {}) or
142                 db.query_candidate_segments(video_id)
143                 if already and not args.reprocess:
144                     print(f"[have] {vid}: detections already in DB (use --reprocess to
145                           redo)")
146                 else:
147                     print(f"[proc] {vid}: segmenting with {args.segmenter}...")
148                     db.add_video(video_id, video_path, "processed", [])
149                     t0 = time.time()
150                     # process_video returns a Result (Success|Failure) for some segmenters
151                     # (e.g. the default `motion`); unwrap before len() so this doesn't crash
152                     with
153                         # TypeError on a Success dataclass ? mirrors the unwrap in
154                         backend/main.py.
155                         segs, failure = unwrap_or_failure(
156                             segmenter.process_video(video_id, video_path,
157                                                       max_frames=args.max_frames))
158                     if failure is not None:

```

```

152             print(f"          -> FAILED: {failure.message}")
153             skipped.append((vid, f"segment failed: {failure.message}"))
154             continue
155             print(f"          -> {len(segs)} segments in {time.time() - t0:.0f}s")
156
157         gt = load_annotations(csv_path)
158         report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
detector=args.detector)
159         print(format_report_text(report, show_failures=args.show_failures))
160         print()
161         d = report_to_dict(report)
162         d["collector_video_id"] = vid
163         per_video.append(d)
164
165     agg = batch.aggregate(per_video, stages)
166     print(batch.format_aggregate(agg))
167     if skipped:
168         print(f"\nSkipped {len(skipped)}: " + ", ".join(f"{v} ({why})" for v, why in
skipped))
169
170     if args.json_out:
171         with open(args.json_out, "w") as f:
172             json.dump({"segmenter": args.segmenter, "iou": args.iou, "stages": stages,
173                 "aggregate": agg, "per_video": per_video,
174                 "skipped": skipped}, f, indent=2)
175             print(f"\nJSON report written to {args.json_out}")
176
177     return 0
178
179
180 if __name__ == "__main__":
181     raise SystemExit(main())
182

```

7. user (2026-06-18T02:00:12.384Z)

```

1     import os
2     import json
3     import logging
4     from dotenv import load_dotenv
5
6     load_dotenv()
7
8     # Central root-logger setup ? all backend modules use logging.getLogger(__name__).
9     # Quiet third-party HTTP chatter by default; re-honoured against the config knob
10    # (logging.verbose_http) once the config is loaded in main().
11    from backend.utils.logging_setup import setup_logging
12    setup_logging()
13
14    from backend.infrastructure.database import Database
15    from backend.config import load_config
16    from backend.utils.hashing import compute_video_id
17    from backend.pipeline.segmenters import segmenter_registry
18    from backend.pipeline.stitcher.compiler import VideoCompiler
19    from backend.results import unwrap_or_failure
20    from backend.tools.export import format_rally_summary
21
22    logger = logging.getLogger(__name__)
23
24    def main():
25        video_path = r"C:\Users\avidu\Videos\Testing\KushagraYashAviFirstVideo.mp4"
26        output_dir = r"C:\Users\avidu\Videos\Testing"
27        output_filename = "Final_2_OutputOfKushagraYashAviFirstVideo.mp4"
28
29        print("Loading config...")

```



```

30     cfg = load_config() # typed AppConfig ? single source of defaults
31     # Re-apply now that the verbose_http knob is known (default keeps HTTP chatter
quiet).
32     setup_logging(verbose_http=cfg.logging.verbose_http)
33
34     print("Verifying files exist...")
35     if not os.path.exists(video_path):
36         print(f"Error: Video file not found at {video_path}")
37         return
38
39     os.makedirs(cfg.output.output_dir, exist_ok=True)
40     db_path = os.path.join(cfg.output.output_dir, cfg.output.db_name)
41     db = Database(db_path)
42
43     # Calculate MD5 of the video to uniquely identify it
44     import time
45     print("Calculating video MD5 (this might take a few seconds on large files)...",
flush=True)
46     t0 = time.time()
47     video_id = compute_video_id(video_path)
48     print(f"Calculated MD5: {video_id} (took {time.time()-t0:.1f}s)", flush=True)
49
50     segments = []
51     cached_video = db.get_video(video_id)
52     cache_hit = False
53
54     if cached_video and cached_video.get("status") == "processed":
55         segments = db.query_play_segments(video_id, {})
56         if len(segments) > 0:
57             print(f"\n[CACHE] Found cached indexed video in database (MD5: {video_id})
with {len(segments)} parsed play segments.", flush=True)
58             choice = input("Would you like to (1) Run stitching on the cached segments,
or (2) Reprocess the video with Gemini from scratch? [1/2]: ").strip()
59             if choice == '1':
60                 print("[CACHE] Using cached play segments. Skipping segmenter API
call.", flush=True)
61                 cache_hit = True
62             else:
63                 print("[CACHE] Ignoring cache...", flush=True)
64         else:
65             print("[CACHE] Video exists in DB but contains 0 play segments. Treating as
cache miss.", flush=True)
66
67     if not cache_hit:
68         print("[CACHE] Initializing clean indexing...", flush=True)
69         db.clear_video_data(video_id)
70
71     print("Bypassing OpenCV validations for performance on massive files...",
flush=True)
72     db.add_video(video_id, video_path, "processed", [])
73
74     print("Running segmenter & indexing...", flush=True)
75     seg_name = cfg.indexing.default_segmenter
76     segmenter_cls = segmenter_registry.get(seg_name)
77     if not segmenter_cls:
78         print(f"[FAIL] Segmenter '{seg_name}' not found.")
79         return
80     print(f"Using segmenter: {seg_name}", flush=True)
81     segmenter = segmenter_cls(db, cfg)
82     # process_video returns a Result (Success|Failure) for some segmenters (e.g.
`motion`);
83     # unwrap before len()/truthiness so this doesn't crash with TypeError on a
Success
84     # dataclass (and so the empty-guard below actually sees the list) ? mirrors
backend/main.py.

```

```

85         segments, failure = unwrap_or_failure(segmenter.process_video(video_id,
video_path))
86         if failure is not None:
87             print(f"[FAIL] Segmentation failed: {failure.message}")
88             return
89         print(f"Successfully indexed {len(segments)} play segments.", flush=True)
90
91     if not segments:
92         print("No play segments detected. Cannot compile highlights.")
93         return
94
95
96     end_padding = cfg.stitching.end_padding
97     begin_padding = cfg.stitching.begin_padding
98     min_shot_count = cfg.stitching.min_shot_count
99
100    if cache_hit:
101        print("\n--- Stitching Customizations ---")
102        try:
103            bpad_input = input(f"Enter begin_padding in seconds (default
{begin_padding}): ").strip()
104            if bpad_input:
105                begin_padding = float(bpad_input)
106
107            pad_input = input(f"Enter end_padding in seconds (default {end_padding}):
").strip()
108            if pad_input:
109                end_padding = float(pad_input)
110
111            shot_input = input(f"Enter min_shot_count to filter by (default
{min_shot_count}): ").strip()
112            if shot_input:
113                min_shot_count = int(shot_input)
114        except ValueError:
115            print("[WARNING] Invalid input detected, falling back to defaults from
config.json.")
116
117    print(f"\nCompiling highlights (with begin_padding={begin_padding}s,
end_padding={end_padding}s, min_shot_count={min_shot_count})...", flush=True)
118
119    # Filter segments where shot_count >= min_shot_count
120    time_pairs = []
121    skipped_count = 0
122    for s in segments:
123        meta = s.get("metadata", {})
124        if isinstance(meta, str):
125            try:
126                meta = json.loads(meta)
127            except Exception:
128                meta = {}
129        shot_count = meta.get("shot_count", 0)
130        if shot_count >= min_shot_count:
131            time_pairs.append((s["start_time"], s["end_time"]))
132        else:
133            skipped_count += 1
134
135    if skipped_count > 0:
136        print(f"[FILTER] Excluded {skipped_count} play segments with fewer than
{min_shot_count} shots.", flush=True)
137    print(f"[FILTER] Retaining {len(time_pairs)} play segments for stitching.",
flush=True)
138
139    # End-of-run rally table for manual QA ? sourced from the produced play_segments so
140    # it lines up row-for-row with the rally CSV. Logged as one block (single timestamp,
141    # aligned rows) so it survives even if the table is the last thing the owner scans.

```

```

142         def _log_rally_summary() -> None:
143             logger.info("\n%s", format_rally_summary(segments))
144
145         if not time_pairs:
146             print("No play segments passed the filtering criteria. Cannot compile
highlights.")
147             _log_rally_summary()
148             return
149
150         compiler = VideoCompiler(output_dir, end_padding=end_padding,
begin_padding=begin_padding,
151                                   watermark=cfg.stitching.watermark)
152         try:
153             out_path = compiler.compile_highlights(video_path, time_pairs, output_filename)
154             print(f"Stitched highlight video saved to: {out_path}", flush=True)
155         except Exception as e:
156             print(f"Error during stitching: {e}", flush=True)
157
158         _log_rally_summary()
159
160     if __name__ == "__main__":
161         main()
162

```

8. assistant (2026-06-18T02:00:15.265Z)

Now let me look at the inline unwrap in backend/main.py that this claims to mirror, to verify the helper is faithful and the Failure-branch semantics match.

9. user (2026-06-18T02:00:17.410Z)

```

35:from backend.results import Failure # standardized error/result contract
192:         result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
195:         # state (same destroy-after-confirm contract as the Failure branch), then let
201:         if isinstance(result, Failure):
213:             segments = result.value if hasattr(result, "value") else result
220:             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
347: def process_video(req: ProcessRequest):
835:         result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
838:         if isinstance(result, Failure):
843:             segments = result.value if hasattr(result, "value") else result

```

10. assistant (2026-06-18T02:00:21.680Z)

Let me read the exact inline unwrap context in backend/main.py to confirm the helper faithfully mirrors it.

11. user (2026-06-18T02:00:22.634Z)

```

190         segmenter = segmenter_cls(db_instance, global_config)
191         try:
192             result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
193         except Exception:
194             # A raising segmenter must not leave half-written rows: restore the pre-run
195             # state (same destroy-after-confirm contract as the Failure branch), then
196             let
197                 # the job worker record the failure.
198                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
199                 raise
200         segmentation_ran = True

```

```

200
201     if isinstance(result, Failure):
202         logger.error(f"[API] Segmenter failed: {result.message}")
203         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
204         response = {
205             "video_id": video_id,
206             "status": "failed",
207             "message": f"Segmenter '{seg_name}' failed: {result.message}",
208             "results": [r.model_dump() for r in val_results],
209             "play_segments": [],
210         }
211         return response
212
213     segments = result.value if hasattr(result, "value") else result
214     notify("finalizing")
215     _finalize_reingest(video_id, segments, play_wm, cand_wm)
216
217     response = {
218         "video_id": video_id,
219         "status": "success",
220         "message": f"Successfully indexed {len(segments)} play segments using
221 '{seg_name}' segmenter.",
222         "results": [r.model_dump() for r in val_results],
223         "play_segments": segments,
224     }
225     return response

```

12. user (2026-06-18T02:00:23.332Z)

```

830
831     print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
832     segmenter_actual = seg_name
833     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
834     segmenter = segmenter_cls(db_instance, global_config)
835     result = segmenter.process_video(video_id, args.video_path,
836 max_frames=args.max_frames)
837     segmentation_ran = True
838
839     if isinstance(result, Failure):
840         print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
841         print("[FAIL] Indexing halted for video.")
842         segments = []
843     else:
844         segments = result.value if hasattr(result, "value") else result
845         print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
846 segments inside database: {db_path}")
847         # Destroy-AFTER-confirm: keep new on success, restore prior on
848 empty/failure.
849         _finalize_reingest(video_id, segments, play_wm, cand_wm)
850     finally:
851         # Always emit the per-video observability report. Never raises.
852         paths = emit_indexer_report(
853             db=db_instance, video_id=video_id, video_path=args.video_path,
854             config=global_config,
855             val_results=val_results, val_context=val_context,
856             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
857             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
858         )

```

13. assistant (2026-06-18T02:00:28.287Z)

The helper faithfully mirrors `backend/main.py:213` / `:843` (`result.value if hasattr(result, "value") else result`, gated by `isinstance(result, Failure)`). Now let me verify the motion segmenter actually returns a `Success`, confirming the bug premise, and check the `process\_video` signature/return contract.

14. user (2026-06-18T02:00:30.560Z)

```
03d11d9 fix(scripts): unwrap the segmenter Result before len()/truthiness (motion crashes the
default path)
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
be39a70 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17) (#206)
4bbd987 docs: rally-detection quality report ? technical-paper checkpoint (2026-06-17)
8736e2d fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels
(#205)
---BRANCH---
fix/scripts-unwrap-segmenter-result
---vs origin/master---
03d11d9 fix(scripts): unwrap the segmenter Result before len()/truthiness (motion crashes the
default path)
```

15. user (2026-06-18T02:00:31.657Z)

```
backend\pipeline\segmenters\base.py:24: def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
backend\pipeline\segmenters\gemini.py:35: def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\motion.py:20: def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\motion.py:31: return Failure(message=f"Could not open
video: {video_path}", is_recoverable=False)
backend\pipeline\segmenters\motion.py:224: return Success(value=segments_data)
backend\pipeline\segmenters\trajectory_hybrid.py:106: def process_video(self, video_id: str,
video_path: str, # type: ignore[override]
backend\pipeline\segmenters\yolo_hybrid.py:57: def process_video(self, video_id: str,
video_path: str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
```

16. assistant (2026-06-18T02:00:38.815Z)

The bug premise is confirmed: `motion.py:224` returns `Success(value=segments\_data)` and `:31` returns `Failure`, while the scripts called `len()/truthiness` directly. Branch is clean (only the one fix commit on top of origin/master).

Let me verify two important edge cases for the Failure-branch semantics:

1. In `run\_phase3\_eval.py`, the `Failure` branch does `continue` ? but note `db.add\_video(...)` was already called at line 144 before segmentation. Let me check whether a half-written video row is left when segmentation fails (potential masking/state issue), and whether that differs from prior behavior.
2. Confirm `Failure` has no `.value` attribute (so the `hasattr` ordering is correct ? `isinstance` check must come first since a Failure has no `.value`, which it doesn't anyway, but order matters for correctness).

Let me also check the gemini segmenter return type, since the `run\_process\_and\_stitch` default segmenter is config-driven.

17. user (2026-06-18T02:00:39.759Z)

```
35 def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
36 """
37 Processes a video to detect play segments using Google Gemini Multimodal Video
API,
38 populates SQLite, and indexes detailed events.
39 """
40 # Get sport profile config
41 sport_name = self.config.get("sport", "badminton")
42 try:
43     sport_profile = sport_registry.get(sport_name)
44 except KeyError as e:
45     logger.error(str(e))
```

```

46         return []
47
48         # Get AI provider and model config
49         idx_cfg = self.config.get("indexing", {})
50         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
51         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
52
53         # Get appropriate API key based on the provider
54         api_key_env_var = f"{provider_name.upper()}_API_KEY"
55         api_key = os.environ.get(api_key_env_var)
56         if not api_key:
57             logger.error(
58                 f"{api_key_env_var} environment variable is not set! "
59                 f"To run the {provider_name} segmenter, set your API key. "
60                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
61             )
62             return []
63
64         try:
65             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
66         except KeyError as e:
67             logger.error(str(e))
68             return []
69
70         # A: Check for debug_duration to trim the video first
71         debug_duration = self.config.get("indexing", {}).get("debug_duration")
72         video_to_upload = video_path
73         is_trimmed = False
74         temp_trimmed_path = None

```

18. user (2026-06-18T02:00:42.695Z)

```

Failure has .value: False
unwrap Failure: (None, Failure(message='x', details=None, is_recoverable=False))
unwrap Success: ([1, 2], None)
unwrap bare: ([1, 2, 3], None)
unwrap None: (None, None)

```

19. assistant (2026-06-18T02:00:49.928Z)

All input types behave correctly. Note: `gemini.py` returns a bare list (`[]`) on its own internal failures (lines 46, 62, 68) rather than a `Failure` ? so a gemini failure would be unwrapped as `([], None)`, treated as "no segments" not "failed". But that is pre-existing behavior identical to `backend/main.py`'s inline unwrap (which also can't distinguish gemini's bare-`[]` failures), so the PR doesn't regress it ? it faithfully mirrors the production contract.

Now let me run the new test file and the full suite to verify green, per the pre-LGTM gate.

20. user (2026-06-18T02:00:51.948Z)

```

.... [100%]
4 passed in 0.06s

```

21. user (2026-06-18T02:02:14.733Z)

```

=====
Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
obsreport (optional reporting dep): available
=====

```

```

.....SS..... [ 8%]
..... [ 16%]
..... [ 25%]
..... [ 33%]
..... [ 42%]
..... [ 50%]
..... [ 58%]
..... [ 67%]
..... [ 75%]
.SSSSS..... [ 84%]
..... [ 92%]
..... [100%]
850 passed, 7 skipped in 73.81s (0:01:13)
=====
RESULT: PASS - all tests green
=====

```

22. assistant (2026-06-18T02:02:17.320Z)

Full suite green (850 passed, 7 skipped). Let me verify lint/type checks per the pre-LGTM gate, focusing on the touched files.

23. user (2026-06-18T02:02:19.929Z)

```

=== RUFF ===
All checks passed!
=== MYPY ===
Success: no issues found in 1 source file

```

24. assistant (2026-06-18T02:02:27.466Z)

Lint and type checks clean. Let me do a final adversarial check on the `run_phase3_eval.py` Failure branch: line 144 calls `db.add_video(...)` **before** segmentation, then on Failure it does `continue` (leaving the video row written but with no segments). I want to verify this isn't a new state-masking regression vs. prior behavior, and whether the prior (crashing) behavior would have left the same row.

Previously, the script would have crashed at `len(segs)` (TypeError) on `Success`, or on `Failure` truthiness ? actually before this fix, the old code `segs = segmenter.process_video(...)` then `len(segs)` would crash for BOTH Success and Failure. So the script was entirely non-functional under `motion`. The new behavior (continue, record skip) is strictly an improvement. The `add_video` row being left on failure is the same as if the prior code had reached that point ? and the loop's score-only/`already` guard at line 139 handles re-runs. No regression.

Let me also confirm there are no OTHER callers of `process_video` in `scripts/` or `tools/` that still have the same latent bug (to assess completeness, not as a blocker).

25. user (2026-06-18T02:02:29.063Z)

```

scripts\run_phase3_eval.py:146:          # process_video returns a Result
(Success|Failure) for some segmenters
scripts\run_phase3_eval.py:150:          segmenter.process_video(video_id, video_path,
max_frames=args.max_frames))
scripts\run_process_and_stitch.py:82:      # process_video returns a Result (Success|Failure)
for some segmenters (e.g. `motion`);
scripts\run_process_and_stitch.py:85:      segments, failure =
unwrap_or_failure(segmenter.process_video(video_id, video_path))

```

26. user (2026-06-18T02:02:30.351Z)

```

scripts\run_phase3_eval.py:150:          segmenter.process_video(video_id, video_path,
max_frames=args.max_frames))
scripts\run_process_and_stitch.py:85:      segments, failure =

```

```

unwrap_or_failure(segmenter.process_video(video_id, video_path))
tools\generate_highlights.py:167:         result = seg_cls(db, cast(Dict[str, Any],
cfg)).process_video(vid, proc_detect)
tests\test_base.py:34:     success = inst.process_video("v1", "path")
tests\test_base.py:40:     failure = inst.process_video("v1", "fail")
tests\test_base.py:192:         inst.process_video("v1", "path")
backend\main.py:192:         result = segmenter.process_video(video_id, req.video_path,
req.player_pool, req.max_frames)
backend\main.py:835:         result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
tests\test_fusion_hybrid.py:55:         res = seg.process_video("v1", "clip.mp4")
tests\test_gemini.py:42:     res = segmenter.process_video("v1", "dummy.mp4")
tests\test_gemini.py:81:     res = segmenter.process_video("v1", "dummy.mp4")
tests\test_gemini.py:108:     return segmenter.process_video(video_id, "dummy.mp4")
tests\test_gemini.py:185:         res = segmenter.process_video("vid_single_oversize",
"dummy.mp4")
tests\test_tracknet_hybrid.py:63:     res = seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:82:     assert seg.process_video("v1", "clip.mp4") == []
tests\test_tracknet_hybrid.py:95:     res = seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:110:     assert seg.process_video("v1", "clip.mp4") == []
tests\test_tracknet_hybrid.py:129:     seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:153:     res = seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:181:     res = seg.process_video("v1", "clip.mp4")
tests\test_trajectory_hybrid_equivalence.py:56:         return db, seg.process_video("v1",
"clip.mp4")
tests\test_yolo_hybrid.py:89:     res = segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:113:     res = segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:156:     res = segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:225:     res = segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:276:     segmenter.process_video("v1", "dummy.mp4")

```

27. assistant (2026-06-18T02:02:33.144Z)

The only two production scripts calling `process\_video` are the two fixed here.
`tools/generate\_highlights.py:167` also calls it ? let me quickly check whether it already handles the Result, to confirm completeness (not a blocker for this PR either way, since it's out of scope).

28. user (2026-06-18T02:02:34.718Z)

```

165         # AppConfig is consumed by segmenters via its dict-compatible .get shim (same as
166         # main.py); cast keeps the type checker honest about that contract.
167         result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid,
proc_detect)
168         if isinstance(result, Failure):
169             logger.error("segmentation FAILED: %s", getattr(result, "message",
result))
170             summary["failed"].append(name)
171             continue
172
173         segs = db.query_play_segments(vid, {})
174         pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
175         logger.info("rallies detected: %d", len(pairs))
176         if not pairs:

```

29. assistant (2026-06-18T02:02:48.370Z)

`tools/generate\_highlights.py` already correctly handles the `Result` (isinstance Failure check + reads from DB, never calls `.value`), so it had no bug. The PR correctly scoped itself to the two broken scripts. It could have also adopted `unwrap\_or\_failure` there for DRY consistency, but that's a nit, not a defect ? and that file doesn't need the `.value` path since it re-queries the DB.

Everything verifies. Summary of my review:

****Correctness (the lens):****

- The bug is real and confirmed: `motion.py:224` returns `Success`, `:31/gemini-failures` vary; the two scripts called `len()/truthiness` on the raw `Success` dataclass which has no `__len__`, crashing the *default* `motion` path. Both `run_phase3_eval.py` and `run_process_and_stitch.py` were fully non-functional under `motion` before this fix.
- `unwrap_or_failure` is a faithful DRY extraction of the inline `unwrap` at `backend/main.py:201/213` and `:838/843` ? identical `isinstance(result, Failure)` gate then `result.value` if `hasattr(result, "value")` else `result`. Verified empirically on `Success` / `Failure` / `bare-list` / `None` inputs.
- Failure-branch semantics are right per script: `run_phase3_eval.py` does `continue` (records skip, moves to next video in a batch loop) ? correct for a batch driver; `run_process_and_stitch.py` does `return` (single-video script, nothing else to do) ? correct. Both strictly improve on the prior universal crash.
- The `add_video` row written before a failed segment in `run_phase3_eval.py` is pre-existing and guarded by the `already/--reprocess` re-run logic; no new state-masking regression.

****Verification (pre-LGTM gate):**** full suite green (850 passed, 7 skipped), new `tests/test_results.py` green (4 passed), `ruff check` clean on all touched files, `mypy backend/results.py` clean.

****Non-blocking observations (nits):****

- `tools/generate_highlights.py:167-171` already handles the `Result` correctly (`isinstance + DB re-query`, no `.value`), so it wasn't broken ? but it could adopt `unwrap_or_failure` for consistency. Not in scope.
- `gemini`'s internal-failure path returns a bare `[]` (not a `Failure`), so a `gemini` failure unwraps as `"no segments"` rather than `"failed"` ? but this is identical to the production `backend/main.py` contract and not introduced by this PR.

No must-fix issues. The change is correct, faithful to the existing contract, well-tested, and clears the verification gate.

30. user (2026-06-18T02:02:58.359Z)

Output does not match required schema: root: must have required property 'findings'

31. user (2026-06-18T02:03:07.556Z)

Output does not match required schema: root: must have required property 'findings'

32. assistant (2026-06-18T02:03:10.789Z)

The error indicates a structural issue. Let me re-call with the findings array properly formed ? possibly the validator needs the arguments at the top level differently. I'll retry ensuring all required keys are present.

33. user (2026-06-18T02:03:17.128Z)

Output does not match required schema: root: must have required property 'findings'

34. assistant (2026-06-18T02:03:26.604Z)

The validator keeps rejecting despite `findings` being present. The JSON string for findings may be getting mis-parsed. Let me try passing findings as a properly typed array (not a JSON-encoded string) ? the schema expects an actual array value.

35. user (2026-06-18T02:03:30.067Z)

Output does not match required schema: root: must have required property 'findings'

36. assistant (2026-06-18T02:03:33.444Z)

The error persists even with a minimal valid findings array, which suggests the tool input may need findings as a native array rather than a stringified JSON. The harness likely parses my parameter value as a string. Let me try without quotes around the array structure by formatting

it as the parameter expects a raw JSON array literal.

37. user (2026-06-18T02:03:38.322Z)

Output does not match required schema: root: must have required property 'findings'

38. assistant (2026-06-18T02:03:41.997Z)

The validator continues to reject. The error "root: must have required property 'findings'" while I am clearly passing findings suggests the harness is not receiving my `findings` parameter at all ? possibly the array value is being dropped during serialization. Let me try with a single-line, compact findings value and verify all four required top-level keys are spelled exactly.

39. user (2026-06-18T02:03:44.933Z)

Output does not match required schema: root: must have required property 'findings'

40. assistant (2026-06-18T02:03:47.832Z)

The schema validation is persistently failing on `findings` despite it being present. Let me check whether perhaps an empty findings array is what's needed, or whether the issue is something subtle. Let me try with an empty findings array since there are genuinely no actionable findings.

41. user (2026-06-18T02:03:51.761Z)

Output does not match required schema: root: must have required property 'findings'

42. assistant (2026-06-18T02:03:57.650Z)

The validator rejects even an empty array. This strongly indicates the harness is dropping the `findings` parameter entirely during transport, regardless of its value. This may be a parsing issue with how the tool call arguments are being serialized. Let me try reordering so `findings` is the first parameter, in case ordering affects serialization.

43. user (2026-06-18T02:04:00.721Z)

Output does not match required schema: root: must have required property 'blocks_merge'

44. assistant (2026-06-18T02:04:05.121Z)

Progress: reordering changed the error from `findings` to `blocks\_merge`. This tells me the harness is only reading the FIRST parameter and dropping the rest. The issue is likely that one of my parameter values contains characters breaking the serialization. The most likely culprit is the long `findings` JSON-in-a-string with nested quotes/special chars confusing the parser. Let me put all parameters but keep findings extremely simple and short to avoid any escaping issues.

45. user (2026-06-18T02:04:07.411Z)

Structured output provided successfully

46. assistant (2026-06-18T02:04:19.463Z)

The structured output was accepted. The issue was parameter ordering and serialization sensitivity in the harness, now resolved.

Review summary: PR #213 ? APPROVE (no must-fix issues)

The fix is correct and clears the verification gate.

****The bug is real and confirmed.**** `backend/pipeline/segmenters/motion.py:224` returns

`Success(value=segments\_data)` and `:31` returns `Failure`. Both `scripts/run\_phase3\_eval.py` and `scripts/run\_process\_and\_stitch.py` called `len()/truthiness` directly on that raw `Result` dataclass (which has no `\_\_len\_\_`), so they crashed with `TypeError` under the **default** **motion** segmenter ? i.e. both scripts were entirely non-functional on the default path.

The helper is faithful. `backend/results.unwrap_or_failure`` is an exact DRY extraction of the inline `unwrap` at `backend/main.py:201/213`` and `:838/843`` ? same `isinstance(result, Failure)`` gate, then `result.value` if `hasattr(result, "value")` else `result``. I verified empirically it returns `(None, failure)`` for `Failure``, `(value, None)`` for `Success``, and `(value, None)`` for a bare list (and `(None, None)`` for `None``).

Failure-branch semantics are right per script. `run_phase3_eval.py`` does `continue`` (correct ? it's a batch loop over videos, records the skip), `run_process_and_stitch.py`` does `return`` (correct ? single-video script). Both are strict improvements over the prior universal crash. The `db.add_video`` row written before a failed segment in `run_phase3_eval.py`` is pre-existing and guarded by the re-run logic ? no new state-masking.

Verification (pre-LGTM gate):

- Full suite: 850 passed, 7 skipped (73.8s)
- New `tests/test_results.py``: 4 passed
- `ruff check`` on all touched files: clean
- `mypy backend/results.py``: clean
- Branch contains only the single fix commit on top of `origin/master``

Nits only (non-blocking): `tools/generate_highlights.py:167`` already handles the `Result`` correctly (`isinstance + DB re-query`) so it wasn't broken ? could adopt the helper for DRY but doesn't need to. The `gemini segmenter`` returns bare `[]`` on its own internal failures rather than a `Failure``, so those surface as "no segments" not "failed" ? but that's pre-existing and identical to the production `backend/main.py`` contract the helper mirrors, not introduced here.

`blocks_merge=false``.